

**LAPORAN PRAKTIKUM 7**  
**STRUKTUR DATA**  
**IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)***  
***SORTING MENGGUNAKAN JAVA***



**Disusun Oleh:**

Naysha Aprilia Rizki

2511531014

**Dosen Pengampu:**

Wahyudi. Dr. S.T.M.T

**Asisten Praktikum:**

Muhammad Galid Avero

**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**

**2026**

## KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 7 Struktur Data: Implementasi Abstract Data Type (ADT) Sorting Menggunakan Bahasa Java” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data **Sorting**, serta mengimplementasikannya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja Insertion Sort, Selection Sort, Bubble Sort dan GUI Insertion Sort.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan **Sorting** dalam pemrograman Java.

Padang, 20 Mei 2026

Penulis

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Dalam dunia pemrograman, pengolahan data secara cepat dan efisien merupakan hal yang sangat penting. Salah satu cara untuk mengelola data adalah dengan menggunakan algoritma pengurutan (sorting). Sorting merupakan proses menyusun data ke dalam urutan tertentu, seperti dari nilai terkecil ke terbesar atau sebaliknya. Dengan data yang sudah terurut, proses pencarian dan pengolahan data dapat dilakukan dengan lebih mudah dan efisien.

Terdapat berbagai macam algoritma sorting yang sering digunakan, di antaranya adalah Bubble Sort, Selection Sort, dan Insertion Sort. Bubble Sort bekerja dengan cara membandingkan dua data yang berdekatan kemudian menukarnya jika urutannya salah. Selection Sort melakukan pengurutan dengan memilih elemen terkecil atau terbesar dari sekumpulan data lalu menempatkannya pada posisi yang sesuai. Sedangkan Insertion Sort bekerja dengan cara menyisipkan data pada posisi yang tepat sehingga bagian data yang telah diproses selalu dalam keadaan terurut.

Masing-masing algoritma memiliki cara kerja, kelebihan, dan kekurangan yang berbeda. Oleh karena itu, pemahaman terhadap algoritma sorting sangat penting bagi mahasiswa agar mampu memilih metode pengurutan yang sesuai dengan kebutuhan program. Selain memahami konsep algoritma, implementasi sorting juga dapat dikembangkan dalam bentuk aplikasi berbasis Graphical User Interface (GUI) menggunakan bahasa pemrograman Java. Dengan GUI, proses pengurutan data dapat ditampilkan secara lebih interaktif sehingga memudahkan pengguna dalam memahami jalannya algoritma sorting.

Pada praktikum ini dilakukan implementasi algoritma Bubble Sort, Selection Sort, dan Insertion Sort menggunakan bahasa pemrograman Java, termasuk penerapan GUI pada Insertion Sort. Praktikum ini bertujuan agar mahasiswa mampu memahami konsep dasar algoritma sorting serta dapat mengimplementasikannya ke dalam program yang interaktif dan mudah digunakan.

### **1.2 Tujuan**

1. Memahami konsep dasar algoritma sorting.
2. Memahami cara kerja Bubble Sort, Selection Sort, dan Insertion Sort.
3. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
4. Membuat aplikasi GUI untuk simulasi Insertion Sort.

5. Melatih kemampuan pemrograman dalam mengelola dan mengurutkan data secara efisien. Melatih kemampuan pemrograman dalam mengelola data secara dinamis menggunakan struktur data.

### **1.3 Manfaat**

1. Menambah pemahaman tentang algoritma pengurutan data (sorting).
2. Membantu memahami perbedaan cara kerja Bubble Sort, Selection Sort, dan Insertion Sort.
3. Melatih kemampuan implementasi algoritma dalam bahasa pemrograman Java.
4. Memberikan pengalaman dalam membuat aplikasi berbasis GUI.
5. Menjadi dasar pembelajaran algoritma dan struktur data yang lebih kompleks.

## **BAB II**

### **PEMBAHASAN**

Dalam pemrograman, algoritma merupakan langkah-langkah sistematis yang digunakan untuk menyelesaikan suatu masalah. Salah satu algoritma yang sering digunakan dalam pengolahan data adalah algoritma pengurutan (sorting). Sorting digunakan untuk menyusun data ke dalam urutan tertentu, seperti urutan menaik (ascending) atau menurun (descending), sehingga data lebih mudah diproses dan dianalisis.

Pada praktikum ini digunakan beberapa algoritma sorting, yaitu Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga algoritma tersebut termasuk algoritma pengurutan sederhana yang sering digunakan sebagai dasar pembelajaran dalam memahami konsep sorting pada pemrograman. Selain itu, implementasi Insertion Sort juga dikembangkan dalam bentuk aplikasi berbasis Graphical User Interface (GUI) menggunakan bahasa pemrograman Java agar proses pengurutan dapat dipahami dengan lebih interaktif.

Dalam bahasa pemrograman Java, implementasi algoritma sorting dilakukan menggunakan array sebagai media penyimpanan data. Array digunakan untuk menyimpan sekumpulan nilai yang nantinya akan diurutkan menggunakan algoritma tertentu. Setiap algoritma memiliki langkah kerja yang berbeda dalam melakukan proses pengurutan data.

#### **1. Bubble Sort**

Bubble Sort merupakan algoritma pengurutan yang bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya apabila urutannya salah. Proses ini dilakukan secara berulang hingga seluruh data berada pada posisi yang benar. Algoritma ini disebut Bubble Sort karena elemen terbesar atau terkecil akan “mengapung” ke posisi akhir seperti gelembung air.

Langkah kerja Bubble Sort:

1. Membandingkan dua elemen yang berdekatan.
2. Menukar posisi elemen jika urutannya salah.
3. Mengulangi proses hingga seluruh data terurut.

Kelebihan Bubble Sort adalah mudah dipahami dan diimplementasikan, namun kekurangannya adalah prosesnya kurang efisien untuk jumlah data yang besar karena membutuhkan banyak perbandingan dan pertukaran data.

## **2. Selection Sort**

Selection Sort merupakan algoritma pengurutan yang bekerja dengan cara mencari elemen terkecil atau terbesar dari bagian array yang belum terurut, kemudian menempatkannya pada posisi yang sesuai. Proses ini dilakukan secara berulang hingga seluruh data terurut.

Langkah kerja Selection Sort:

1. Menentukan elemen terkecil dari array.
2. Menukar elemen tersebut dengan elemen pada posisi awal yang belum terurut.
3. Mengulangi proses pada bagian array berikutnya hingga selesai.

Kelebihan Selection Sort adalah jumlah pertukaran data lebih sedikit dibanding Bubble Sort. Namun, algoritma ini tetap membutuhkan banyak proses perbandingan sehingga kurang efisien untuk data berukuran besar.

## **3. Insertion Sort**

Insertion Sort merupakan algoritma pengurutan yang bekerja dengan cara menyisipkan elemen pada posisi yang tepat dalam bagian array yang sudah terurut. Algoritma ini mirip seperti cara mengurutkan kartu secara manual.

Langkah kerja Insertion Sort:

1. Mengambil satu elemen dari array.
2. Membandingkannya dengan elemen sebelumnya yang sudah terurut.
3. Menggeser elemen yang lebih besar.
4. Menyisipkan elemen pada posisi yang sesuai.
5. Mengulangi proses hingga seluruh data terurut.

Insertion Sort memiliki kelebihan dalam pengurutan data berukuran kecil karena lebih cepat dan sederhana. Selain itu, algoritma ini juga mudah dipahami dalam proses implementasi program. Namun, untuk data berukuran besar, performanya masih kurang efisien dibanding algoritma sorting yang lebih kompleks.

## **4. GUI Insertion Sort**

Graphical User Interface (GUI) merupakan tampilan antarmuka visual yang memungkinkan pengguna berinteraksi dengan program secara lebih mudah. Pada

praktikum ini, algoritma Insertion Sort diimplementasikan dalam bentuk GUI menggunakan Java Swing.

GUI Insertion Sort dibuat agar pengguna dapat memasukkan data, menjalankan proses sorting, dan melihat hasil pengurutan secara langsung melalui tampilan visual. Komponen GUI yang digunakan antara lain:

1. JFrame sebagai jendela utama program.
2. JTextField untuk input data.
3. JButton untuk menjalankan proses sorting.
4. JTextArea untuk menampilkan hasil pengurutan.

Dengan adanya GUI, proses pengurutan data menjadi lebih interaktif dan mudah dipahami dibandingkan hanya menggunakan tampilan console. Selain itu, implementasi GUI juga melatih kemampuan mahasiswa dalam mengembangkan aplikasi desktop menggunakan bahasa pemrograman Java.

Melalui praktikum ini, mahasiswa dapat memahami konsep dasar algoritma sorting, mengetahui perbedaan cara kerja masing-masing algoritma, serta mampu mengimplementasikan Bubble Sort, Selection Sort, dan Insertion Sort ke dalam program Java, baik berbasis console maupun GUI.

## 1. Langkah Kerja

### 1.1 Insertion Sort

- Kode Program

```
1 package pekan7_2511531014;
2
3 public class InsertionSort_2511531014 {
4     public static void insertionSort1014(int[] arr_1014) {
5         int n_1014 = arr_1014.length;
6         for (int i_1014 = 1; i_1014 < n_1014; i_1014++) {
7             int key_1014 = arr_1014[i_1014];
8             int j_1014 = i_1014 - 1;
9
10            while (j_1014 >= 0 && arr_1014[j_1014] > key_1014) {
11                arr_1014[j_1014 + 1] = arr_1014[j_1014];
12                j_1014--;
13            }
14            arr_1014[j_1014 + 1] = key_1014;
15        }
16    }
17    public static void main(String[] args_1014) {
18        int arr_1014[] = {23, 78, 45, 8, 32, 56, 1};
19        int n_1014 = arr_1014.length;
20
21        System.out.printf("array yang belum terurut:\n");
22        for (int i_1014 = 0; i_1014 < n_1014; i_1014++)
23            System.out.print(arr_1014[i_1014] + " ");
24
25        System.out.println("");
26
27        insertionSort1014(arr_1014);
28
29        System.out.printf("array yang terurut:\n");
30        for (int i_1014 = 0; i_1014 < n_1014; i_1014++)
31            System.out.print(arr_1014[i_1014] + " ");
32
33        System.out.println("");
34    }
35 }
```

## • Pembahasan

Program ini dibuat dengan mendefinisikan class InsertionSort\_2511531014 yang digunakan untuk mengimplementasikan algoritma Insertion Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending).

1. Class ini memiliki method insertionSort1014(int[] arr\_1014) yang digunakan untuk menjalankan proses pengurutan data menggunakan algoritma Insertion Sort. Method ini menerima parameter berupa array integer yang akan diurutkan.
2. Pada method insertionSort1014(int[] arr\_1014), variabel n\_1014 digunakan untuk menyimpan jumlah elemen pada array. Proses sorting dilakukan menggunakan perulangan for yang dimulai dari indeks ke-1 karena elemen pertama dianggap sudah berada pada posisi yang benar.

```
1 package pekan7_2511531014;
2
3 public class InsertionSort_2511531014 {
4     public static void insertionSort1014(int[] arr_1014) {
5         int n_1014 = arr_1014.length;
```

3. Pada setiap iterasi, program menyimpan nilai elemen yang sedang diproses ke dalam variabel key\_1014. Variabel ini digunakan sebagai data yang akan disisipkan ke posisi yang sesuai pada bagian array yang sudah terurut.

```
int n_1014 = arr_1014.length;
for (int i_1014 = 1; i_1014 < n_1014; i_1014++) {
    int key_1014 = arr_1014[i_1014];
    int j_1014 = i_1014 - 1;
```

4. Variabel j\_1014 digunakan untuk membandingkan elemen sebelumnya dengan nilai key\_1014. Selama elemen sebelumnya lebih besar dari key\_1014, maka elemen tersebut akan digeser satu posisi ke kanan menggunakan perulangan while.

```
while (j_1014 >= 0 && arr_1014[j_1014] > key_1014) {
    arr_1014[j_1014 + 1] = arr_1014[j_1014];
    j_1014--;
}
```

5. Setelah ditemukan posisi yang tepat, nilai key\_1014 dimasukkan kembali ke dalam array pada indeks j\_1014 + 1. Proses ini dilakukan berulang hingga seluruh elemen array berhasil diurutkan secara ascending.

```
    }
    arr_1014[j_1014 + 1] = key_1014;
}
```

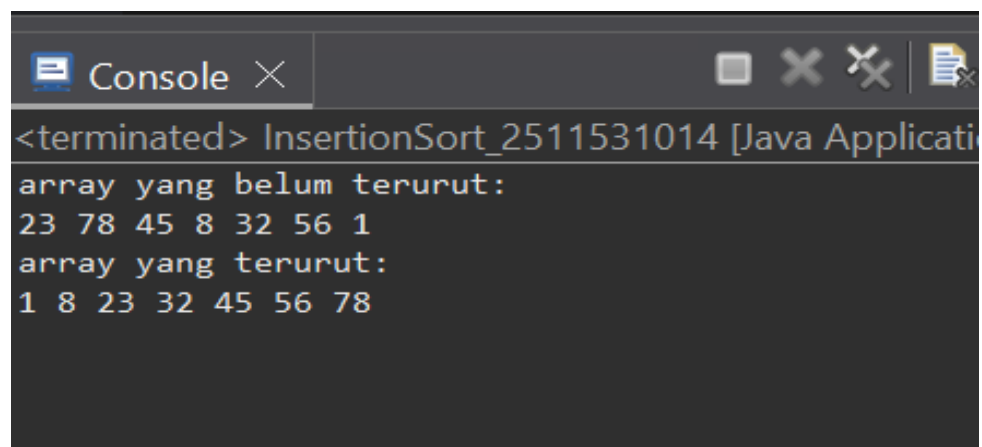
6. Pada method main, program membuat array integer yang berisi beberapa data angka yang belum terurut. Data array kemudian ditampilkan menggunakan perulangan for untuk menunjukkan kondisi awal array sebelum proses sorting dilakukan.

```
public static void main(String[] args_1014) {  
    int arr_1014[] = {23, 78, 45, 8, 32, 56, 1};  
    int n_1014 = arr_1014.length;  
  
    System.out.printf("array yang belum terurut:\n");  
    for (int i_1014 = 0; i_1014 < n_1014; i_1014++)  
        System.out.print(arr_1014[i_1014] + " ");  
  
    System.out.println("");
```

7. Selanjutnya, method `insertionSort1014(arr_1014)` dipanggil untuk menjalankan proses pengurutan menggunakan algoritma Insertion Sort. Setelah proses sorting selesai, isi array kembali ditampilkan untuk menunjukkan hasil array yang sudah terurut secara ascending.

```
5         System.out.println("");  
6  
7         insertionSort1014(arr_1014);  
8  
9         System.out.printf("array yang terurut:\n");  
0         for (int i_1014 = 0; i_1014 < n_1014; i_1014++)  
1             System.out.print(arr_1014[i_1014] + " ");  
2  
3         System.out.println("");  
4     }  
5 }
```

8. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Insertion Sort dalam mengurutkan data integer dengan cara menyisipkan elemen ke posisi yang sesuai pada bagian array yang telah terurut hingga seluruh data tersusun dengan benar.



```
<terminated> InsertionSort_2511531014 [Java Applicati  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

## 1.2 Selection Sort

### • Kode Program

```
1 package pekan7_2511531014;
2
3 public class SelectionSort_2511531014 {
4     public static void selectionSort_2511531014(int[] arr) {
5         int n_1014 = arr.length;
6         for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
7             int minIndex = i_1014;
8             for (int j_1014 = i_1014 + 1; j_1014 < n_1014; j_1014++) {
9                 if (arr[j_1014] < arr[minIndex]) {
10                     minIndex = j_1014;
11                 }
12             }
13             int temp = arr[i_1014];
14             arr[i_1014] = arr[minIndex];
15             arr[minIndex] = temp;
16         }
17     }
18     public static void main(String[] args) {
19         int arr[] = {23, 78, 45, 8, 32, 56, 1};
20         int n_1014 = arr.length;
21         System.out.printf("array yang belum terurut:\n");
22         for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
23             System.out.print(arr[i_1014] + " ");
24         }
25         System.out.println("");
26         selectionSort_2511531014(arr);
27         System.out.printf("array yang terurut:\n");
28         for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
29             System.out.print(arr[i_1014] + " ");
30         }
31         System.out.println("");
32     }
33 }
```

### • Pembahasan

Program ini dibuat dengan mendefinisikan class SelectionSort\_2511531014 yang digunakan untuk mengimplementasikan algoritma Selection Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending).

1. Class ini memiliki method selectionSort\_2511531014(int[] arr) yang digunakan untuk melakukan proses pengurutan data menggunakan algoritma Selection Sort. Method ini menerima parameter berupa array integer yang akan diurutkan.

```
package pekan7_2511531014;

public class SelectionSort_2511531014 {
    public static void selectionSort_2511531014(int[] arr) {
```

2. Pada method selectionSort\_2511531014(int[] arr), variabel n\_1014 digunakan untuk menyimpan panjang array. Proses sorting dilakukan menggunakan dua buah perulangan, yaitu perulangan luar untuk menentukan posisi elemen saat ini dan perulangan dalam untuk mencari indeks nilai terkecil pada bagian array yang belum terurut.

```
    int n_1014 = arr.length;
    for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
```

3. Variabel minIndex\_1014 digunakan untuk menyimpan indeks elemen terkecil yang ditemukan selama proses pencarian. Jika ditemukan elemen

yang lebih kecil dari elemen pada indeks minimum saat ini, maka nilai minIndex\_1014 akan diperbarui sesuai posisi elemen tersebut.

```
int minIndex = i_1014;
for (int j_1014 = i_1014 + 1; j_1014 < n_1014; j_1014++) {
    if (arr[j_1014] < arr[minIndex]) {
        minIndex = j_1014;
    }
}
```

- Setelah nilai terkecil ditemukan, program melakukan proses pertukaran data (swap) menggunakan variabel sementara temp\_1014. Nilai pada indeks saat ini akan ditukar dengan nilai terkecil yang ditemukan sehingga elemen terkecil berpindah ke posisi yang benar.

```
int temp = arr[i_1014];
arr[i_1014] = arr[minIndex];
arr[minIndex] = temp;
}
```

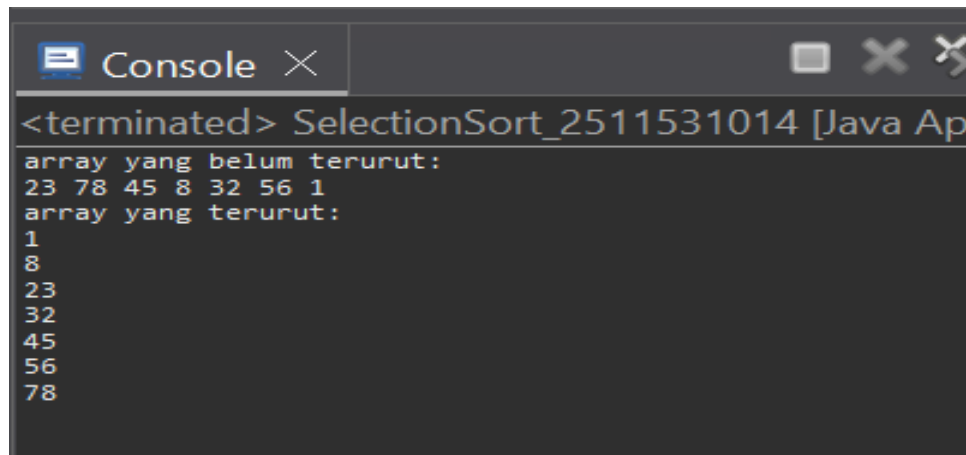
- Pada method main, program membuat array integer yang berisi beberapa data angka yang belum terurut. Data array kemudian ditampilkan menggunakan perulangan for untuk menunjukkan kondisi awal array sebelum proses sorting dilakukan.

```
public static void main(String[] args) {
    int arr[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1014 = arr.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
        System.out.print(arr[i_1014] + " ");
    }
}
```

- Selanjutnya, method selectionSort\_2511531014(arr) dipanggil untuk menjalankan proses pengurutan data menggunakan algoritma Selection Sort. Setelah proses sorting selesai, isi array kembali ditampilkan untuk menunjukkan hasil array yang sudah terurut secara ascending.

```
System.out.println("");
selectionSort_2511531014(arr);
System.out.printf("array yang terurut:\n");
for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
    System.out.print(arr[i_1014] + " ");
}
System.out.println("");
}
```

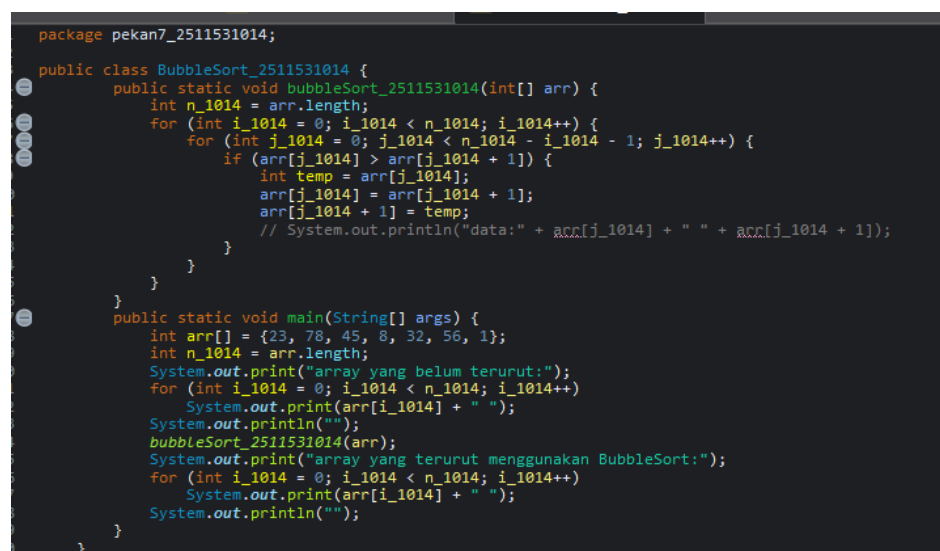
- Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Selection Sort dalam mengurutkan data integer dengan cara mencari elemen terkecil pada setiap iterasi kemudian menukarnya ke posisi yang sesuai hingga seluruh data terurut dengan benar.



```
<terminated> SelectionSort_2511531014 [Java Ap
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1
8
23
32
45
56
78
```

### 1.3 Bubble Sort

- Kode Program



```
package pekan7_2511531014;

public class BubbleSort_2511531014 {
    public static void bubbleSort_2511531014(int[] arr) {
        int n_1014 = arr.length;
        for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
            for (int j_1014 = 0; j_1014 < n_1014 - i_1014 - 1; j_1014++) {
                if (arr[j_1014] > arr[j_1014 + 1]) {
                    int temp = arr[j_1014];
                    arr[j_1014] = arr[j_1014 + 1];
                    arr[j_1014 + 1] = temp;
                    // System.out.println("data:" + arr[j_1014] + " " + arr[j_1014 + 1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = {23, 78, 45, 8, 32, 56, 1};
        int n_1014 = arr.length;
        System.out.print("array yang belum terurut:");
        for (int i_1014 = 0; i_1014 < n_1014; i_1014++)
            System.out.print(arr[i_1014] + " ");
        System.out.println("");
        bubbleSort_2511531014(arr);
        System.out.print("array yang terurut menggunakan BubbleSort:");
        for (int i_1014 = 0; i_1014 < n_1014; i_1014++)
            System.out.print(arr[i_1014] + " ");
        System.out.println("");
    }
}
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class BubbleSort\_2511531014 yang digunakan untuk mengimplementasikan algoritma Bubble Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending).

1. Class ini memiliki method bubbleSort\_2511531014(int[] arr) yang digunakan untuk menjalankan proses pengurutan data menggunakan algoritma Bubble Sort. Method ini menerima parameter berupa array integer yang akan diurutkan.

```
package pekan7_2511531014;

public class BubbleSort_2511531014 {
    public static void bubbleSort_2511531014(int[] arr) {
        int n_1014 = arr.length;
```

2. Pada method bubbleSort\_2511531014(int[] arr), variabel n\_1014 digunakan untuk menyimpan jumlah elemen pada array. Proses sorting dilakukan menggunakan dua buah perulangan for, yaitu perulangan luar untuk menentukan jumlah tahap pengurutan dan perulangan dalam untuk membandingkan elemen array yang bersebelahan.

```
for (int i_1014 = 0; i_1014 < n_1014; i_1014++) {
    for (int j_1014 = 0; j_1014 < n_1014 - i_1014 - 1; j_1014++) {
        if (arr[j_1014] > arr[j_1014 + 1]) {
            int temp = arr[j_1014];
```

3. Pada setiap proses perbandingan, program memeriksa apakah nilai pada indeks saat ini lebih besar dibandingkan nilai pada indeks berikutnya menggunakan kondisi:

if (arr[j\_1014] > arr[j\_1014 + 1])

Jika kondisi tersebut bernilai benar, maka dilakukan proses pertukaran data (swap) agar elemen yang lebih kecil berpindah ke depan.

```
arr[j_1014] = arr[j_1014 + 1];
arr[j_1014 + 1] = temp;
// System.out.println("data:" + arr[j_1014] + " " + arr[j_1014 + 1]);
    }
}
}
```

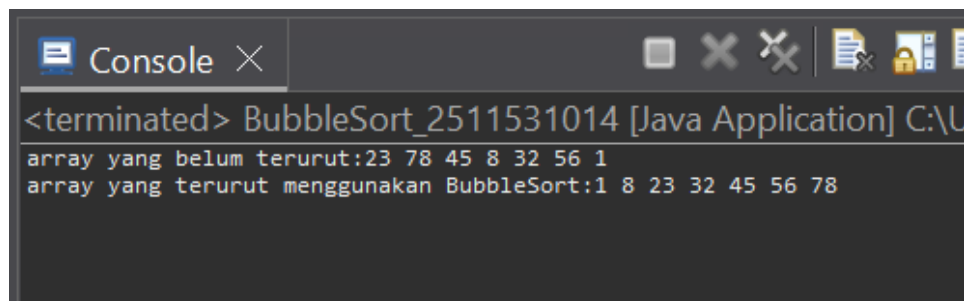
4. Proses pertukaran data dilakukan menggunakan variabel sementara temp. Nilai elemen pertama disimpan sementara ke dalam variabel temp, kemudian posisi kedua elemen ditukar sehingga urutan data menjadi lebih kecil ke besar.
5. Pada setiap iterasi, nilai terbesar akan berpindah ke bagian akhir array. Proses ini dilakukan berulang kali hingga seluruh data berada pada posisi yang benar dan array menjadi terurut secara ascending.
6. Pada method main, program membuat array integer yang berisi beberapa data angka yang belum terurut. Data array kemudian ditampilkan menggunakan perulangan for untuk menunjukkan kondisi awal array sebelum dilakukan proses sorting.

```
public static void main(String[] args) {
    int arr[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1014 = arr.length;
    System.out.print("array yang belum terurut:");
    for (int i_1014 = 0; i_1014 < n_1014; i_1014++)
        System.out.print(arr[i_1014] + " ");
    System.out.println("");
    bubbleSort_2511531014(arr);
}
```

7. Selanjutnya, method `bubbleSort_2511531014(arr)` dipanggil untuk menjalankan proses pengurutan menggunakan algoritma Bubble Sort. Setelah proses sorting selesai, isi array kembali ditampilkan untuk menunjukkan hasil array yang sudah terurut secara ascending.

```
        System.out.print("array yang terurut menggunakan BubbleSort:");  
        for (int i_1014 = 0; i_1014 < n_1014; i_1014++)  
            System.out.print(arr[i_1014] + " ");  
        System.out.println("");  
    }  
}
```

8. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Bubble Sort dalam mengurutkan data integer dengan metode membandingkan dua elemen yang berdekatan kemudian menukarnya secara berulang hingga seluruh data terurut dengan benar.



```
<terminated> BubbleSort_2511531014 [Java Application] C:\U  
array yang belum terurut:23 78 45 8 32 56 1  
array yang terurut menggunakan BubbleSort:1 8 23 32 45 56 78
```

## 1.4 GUI Insertion Sort

- Kode Program

```
1 package pekan7_2511531014;  
2  
3 import java.awt.BorderLayout;  
4 import java.awt.FlowLayout;  
5  
6 import javax.swing.JFrame;  
7 import javax.swing.JPanel;  
8 import javax.swing.border.EmptyBorder;  
9 import javax.swing.JLabel;  
10 import javax.swing.JButton;  
11 import javax.swing.JTextField;  
12 import javax.swing.JTextArea;  
13 import java.awt.Font;  
14 import javax.swing.JScrollPane;  
15 import java.awt.Color;  
16 import java.awt.Dimension;  
17  
18 import javax.swing.BorderFactory;  
19 import javax.swing.JOptionPane;  
20 import javax.swing.SwingConstants;  
21  
22 public class GUIinsertionsort_2511531014 extends JFrame {  
23  
24     private static final long serialVersionUID = 1L;  
25  
26     private JPanel contentPane;  
27     private int[] array_1014;  
28     private JLabel[] labelArray_1014;  
29     private JButton stepButton_1014, resetButton_1014, setButton_1014;  
30     private JTextField inputField_1014;  
31     private JPanel panelArray_1014;  
32     private JTextArea stepArea_1014;  
33  
34     private int i = 1, j;  
35     private boolean sorting_1014 = false;  
36     private int stepCount_1014 = 1;  
37
```

```

38 public GUIinsertionsort_2511531014() {
39
40     setTitle("Insertion Sort Langkah per Langkah");
41     setSize(750, 400);
42     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43     setLocationRelativeTo(null);
44     setLayout(new BorderLayout());
45
46     // Panel input
47     JPanel inputPanel = new JPanel(new FlowLayout());
48
49     inputField_1014 = new JTextField(30);
50     setButton_1014 = new JButton("Set Array");
51
52     inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
53     inputPanel.add(inputField_1014);
54     inputPanel.add(setButton_1014);
55
56     // Panel array visual
57     panelArray_1014 = new JPanel();
58     panelArray_1014.setLayout(new FlowLayout());
59
60     // Panel kontrol
61     JPanel controlPanel = new JPanel();
62     stepButton_1014 = new JButton("Langkah Selanjutnya");
63     resetButton_1014 = new JButton("Reset");
64     stepButton_1014.setEnabled(false);
65     controlPanel.add(stepButton_1014);
66     controlPanel.add(resetButton_1014);
67
68     //Area teks untuk log langkah langkah
69     stepArea_1014 = new JTextArea(0, 60);
70     stepArea_1014.setEditable(false);
71     stepArea_1014.setFont(new Font("Monospaced", Font.PLAIN, 14));
72     JScrollPane scrollPane = new JScrollPane(stepArea_1014);
73
74     // Tambahkan panel ke frame
75     add(inputPanel, BorderLayout.NORTH);
76     add(panelArray_1014, BorderLayout.CENTER);
77     add(controlPanel, BorderLayout.SOUTH);
78     add(scrollPane, BorderLayout.EAST);
79
80     // Event Set Array
81     setButton_1014.addActionListener(e -> setArrayFromInput());
82
83     // Event Langkah Selanjutnya
84     stepButton_1014.addActionListener(e -> performStep());
85
86     // Event Reset
87     resetButton_1014.addActionListener(e -> reset());
88 }

```

```

90 private void setArrayFromInput() {
91     String text = inputField_1014.getText().trim();
92     if (text.isEmpty())
93         return;
94     String[] parts = text.split(",");
95     array_1014 = new int[parts.length];
96     try {
97         for (int k = 0; k < parts.length; k++) {
98             array_1014[k] = Integer.parseInt(parts[k].trim());
99         }
100     } catch (NumberFormatException e) {
101         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
102             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
103         return;
104     }
105     i = 1;
106     stepCount_1014 = 1;
107     sorting_1014 = true;
108     stepButton_1014.setEnabled(true);
109     stepArea_1014.setText("");
110     panelArray_1014.removeAll();
111     labelArray_1014 = new JLabel[array_1014.length];
112     for (int k = 0; k < array_1014.length; k++) {
113         labelArray_1014[k] = new JLabel(String.valueOf(array_1014[k]));
114         labelArray_1014[k].setFont(new Font("Arial", Font.BOLD, 24));
115         labelArray_1014[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
116         labelArray_1014[k].setPreferredSize(new Dimension(50, 50));
117         labelArray_1014[k].setHorizontalAlignment(SwingConstants.CENTER);
118         panelArray_1014.add(labelArray_1014[k]);
119     }
120     panelArray_1014.revalidate();
121     panelArray_1014.repaint();
122 }
123
124 private void performStep() {
125     if (i < array_1014.length && sorting_1014) {
126         int key = array_1014[i];
127         j = i - 1;
128
129         StringBuilder stepLog = new StringBuilder();
130         stepLog.append("Langkah ").append(stepCount_1014)
131             .append(": Memasukkan ").append(key).append("\n");
132
133         while (j >= 0 && array_1014[j] > key) {
134             array_1014[j + 1] = array_1014[j];
135             j--;
136         }
137
138         array_1014[j + 1] = key;
139
140         updateLabels();
141         stepLog.append("Hasil: ") .append(arrayToString(array_1014)).append("\n\n");
142         stepArea_1014.append(stepLog.toString());
143
144         i++;
145         stepCount_1014++;
146
147         if (i >= array_1014.length) {
148             sorting_1014 = false;
149             stepButton_1014.setEnabled(false);
150             JOptionPane.showMessageDialog(this, "Sorting selesai!");
151         }
152     }
153
154     private void updateLabels() {
155         for (int k = 0; k < array_1014.length; k++) {
156             labelArray_1014[k].setText(String.valueOf(array_1014[k]));
157         }
158     }
159
160     private void reset() {
161         inputField_1014.setText("");
162         panelArray_1014.removeAll();
163         panelArray_1014.repaint();
164         panelArray_1014.revalidate();
165         stepArea_1014.setText("");
166         stepButton_1014.setEnabled(false);
167         sorting_1014 = false;
168         i = 1;
169         stepCount_1014 = 1;
170     }
171
172     private String arrayToString(int[] arr) {
173         StringBuilder sb = new StringBuilder();
174         for (int k = 0; k < arr.length; k++) {
175             sb.append(arr[k]);
176             if (k < arr.length - 1) {
177                 sb.append(", ");
178             }
179         }
180         return sb.toString();
181     }
182
183     public static void main(String[] args) {
184         javax.swing.SwingUtilities.invokeLater(() -> {
185             GUIinsertionsort_2511531014 gui =
186                 new GUIinsertionsort_2511531014();
187             gui.setVisible(true);
188         });
189     }
190 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class `GUIinsertionsort_2511531014` yang digunakan untuk mengimplementasikan algoritma Insertion Sort secara visual menggunakan Java GUI (Swing). Class ini berfungsi untuk menampilkan proses pengurutan data langkah demi langkah sehingga pengguna dapat memahami cara kerja algoritma Insertion Sort dengan lebih mudah.

1. Class ini merupakan turunan dari `JFrame` yang digunakan sebagai window utama pada aplikasi GUI. Program memanfaatkan beberapa komponen Swing seperti `JPanel`, `JButton`, `JTextField`, `JLabel`, dan `JTextArea` untuk membangun tampilan antarmuka program.
2. Class ini memiliki atribut `int[] array_1014` yang digunakan untuk menyimpan data array integer yang akan diurutkan. Selain itu, terdapat atribut `JLabel[] labelArray_1014` yang digunakan untuk menampilkan visualisasi elemen array pada GUI.
3. Program juga memiliki beberapa tombol yaitu `setButton_1014`, `stepButton_1014`, dan `resetButton_1014`. Tombol Set Array digunakan untuk memasukkan data array dari input pengguna, tombol Langkah Selanjutnya digunakan untuk menjalankan proses sorting satu langkah demi satu langkah, sedangkan tombol Reset digunakan untuk mengembalikan program ke kondisi awal.
4. Method `setArrayFromInput()` digunakan untuk membaca input angka yang dipisahkan dengan koma dari `JTextField`. Input tersebut kemudian dipecah menggunakan method `split(",")` dan dikonversi menjadi array integer. Jika input tidak valid, program akan menampilkan pesan error menggunakan `JOptionPane`.
5. Setelah data berhasil dimasukkan, program membuat visualisasi array menggunakan `JLabel`. Setiap elemen array ditampilkan dalam bentuk kotak sehingga memudahkan pengguna melihat perubahan posisi data selama proses sorting berlangsung.
6. Method `performStep()` digunakan untuk menjalankan algoritma Insertion Sort secara bertahap. Pada method ini, program mengambil satu elemen sebagai key, kemudian membandingkannya dengan elemen sebelumnya. Jika elemen sebelumnya lebih besar, maka data akan digeser ke kanan hingga posisi yang sesuai ditemukan.
7. Setelah proses penyisipan selesai, array diperbarui dan hasil langkah sorting ditampilkan pada `JTextArea` menggunakan variabel `stepLog`. Proses ini dilakukan setiap kali pengguna menekan tombol Langkah Selanjutnya.

8. Method `updateLabels()` digunakan untuk memperbarui tampilan visual array pada GUI sesuai hasil sorting terbaru. Nilai pada setiap `JLabel` akan diubah mengikuti isi array yang telah diproses.
9. Method `reset()` digunakan untuk menghapus seluruh data input, membersihkan panel visualisasi array, menghapus log langkah sorting, serta mengembalikan variabel program ke kondisi awal sehingga pengguna dapat melakukan proses sorting kembali dari awal.
10. Method `arrayToString(int[] arr)` digunakan untuk mengubah isi array menjadi bentuk string agar hasil proses sorting dapat ditampilkan dengan lebih rapi pada `JTextArea`.
11. Pada method `main`, program dijalankan menggunakan `SwingUtilities.invokeLater()` agar proses pembuatan GUI berjalan pada event dispatch thread sehingga tampilan GUI dapat berjalan dengan stabil.
12. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Insertion Sort melalui visualisasi GUI interaktif. Pengguna dapat melihat proses pengurutan data secara bertahap mulai dari kondisi array belum terurut hingga seluruh data berhasil diurutkan secara ascending.

## **KESIMPULAN**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi algoritma sorting menggunakan bahasa pemrograman Java telah berhasil dilakukan dengan baik. Algoritma yang digunakan dalam praktikum ini meliputi Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga algoritma tersebut digunakan untuk mengurutkan data dari nilai terkecil ke terbesar (ascending).

Melalui program yang telah dibuat, dapat dipahami bahwa setiap algoritma sorting memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. Bubble Sort bekerja dengan membandingkan dua elemen yang berdekatan kemudian menukarnya apabila urutannya salah. Selection

Sort bekerja dengan memilih elemen terkecil dari bagian array yang belum terurut lalu menempatkannya pada posisi yang sesuai. Sedangkan Insertion Sort bekerja dengan cara menyisipkan elemen pada posisi yang tepat dalam bagian array yang sudah terurut.

Dari hasil implementasi dan pengujian program, dapat disimpulkan bahwa algoritma sorting mampu membantu proses pengelolaan data menjadi lebih teratur dan mudah diproses. Selain itu, penggunaan array dalam implementasi sorting mempermudah proses penyimpanan dan pengurutan data secara sistematis.

Pada praktikum ini juga dilakukan implementasi GUI Insertion Sort menggunakan Java Swing. Dengan adanya GUI, proses pengurutan data dapat ditampilkan secara lebih interaktif sehingga pengguna dapat melihat hasil sorting dengan lebih mudah dibandingkan tampilan console biasa. Penerapan GUI juga membantu meningkatkan pemahaman mengenai hubungan antara algoritma dan tampilan aplikasi dalam pemrograman Java.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep algoritma sorting, perbedaan cara kerja Bubble Sort, Selection Sort, dan Insertion Sort, serta penerapannya dalam bahasa pemrograman Java baik berbasis console maupun GUI. Praktikum ini juga membantu meningkatkan kemampuan dalam merancang program pengurutan data secara efisien dan terstruktur.

## **TUGAS PRAKTIKUM**

**Topik : Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort) Tema : Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI**

NAMA CLASS Mahasiswa\_2511531014

- **Kode Program**

```

1 package tugaspekan7_2511531014;
2
3 public class Mahasiswa_2511531014 {
4
5     private String nama_1014;
6     private String nim_1014;
7     private String prodi_1014;
8
9     // Constructor
10    public Mahasiswa_2511531014(String nama_1014, String nim_1014, String prodi_1014) {
11        this.nama_1014 = nama_1014;
12        this.nim_1014 = nim_1014;
13        this.prodi_1014 = prodi_1014;
14    }
15
16    // Getter dan Setter
17    public String getNama() {
18        return nama_1014;
19    }
20
21    public void setNama(String nama) {
22        this.nama_1014 = nama;
23    }
24
25    public String getNim() {
26        return nim_1014;
27    }
28
29    public void setNim(String nim) {
30        this.nim_1014 = nim;
31    }
32
33    public String getProdi() {
34        return prodi_1014;
35    }
36
37    public void setProdi(String prodi) {
38        this.prodi_1014 = prodi;
39    }
40
41    // toString
42    @Override
43    public String toString() {
44        return nama_1014 + " - " + nim_1014 + " - " + prodi_1014;
45    }
46 }
47

```

## - Pembahasan

Program ini dibuat dengan mendefinisikan class Mahasiswa\_2511531014 yang digunakan untuk merepresentasikan data mahasiswa menggunakan konsep Pemrograman Berorientasi Objek (Object Oriented Programming/OOP) dalam bahasa Java. Class ini berfungsi untuk menyimpan data mahasiswa seperti nama, NIM, dan program studi serta menyediakan method untuk mengakses dan mengubah data tersebut.

1. Class Mahasiswa\_2511531014 merupakan class utama yang digunakan sebagai blueprint atau cetakan objek mahasiswa. Class ini dibuat menggunakan keyword public class sehingga dapat diakses oleh class lain dalam package yang sama maupun berbeda.
2. Class ini memiliki tiga atribut bertipe String, yaitu nama\_1014, nim\_1014, dan prodi\_1014. Ketiga atribut tersebut digunakan untuk menyimpan informasi mahasiswa berupa nama, nomor induk mahasiswa, dan program studi. Semua atribut dideklarasikan menggunakan access modifier private agar data terlindungi dan hanya dapat diakses melalui method tertentu.
3. Program memiliki constructor Mahasiswa\_2511531014(String nama\_1014, String nim\_1014, String prodi\_1014) yang digunakan untuk

menginisialisasi objek saat pertama kali dibuat. Constructor menerima parameter nama, NIM, dan prodi kemudian menyimpannya ke atribut class menggunakan keyword this.

4. Method `getNama()` digunakan untuk mengambil atau mengembalikan nilai atribut `nama_1014`. Method ini termasuk getter method yang berfungsi untuk membaca data nama mahasiswa dari luar class.
5. Method `setNama(String nama)` digunakan untuk mengubah nilai atribut `nama_1014`. Method ini termasuk setter method yang memungkinkan pengguna memperbarui data nama mahasiswa.
6. Method `getNim()` digunakan untuk mengambil nilai atribut `nim_1014`, sedangkan method `setNim(String nim)` digunakan untuk mengubah data NIM mahasiswa. Kedua method ini berfungsi sebagai getter dan setter untuk atribut NIM.
7. Method `getProdi()` digunakan untuk mengambil data program studi mahasiswa, sedangkan method `setProdi(String prodi)` digunakan untuk mengubah data program studi yang tersimpan pada atribut `prodi_1014`.
8. Program juga memiliki method `toString()` yang digunakan untuk mengubah objek mahasiswa menjadi bentuk string. Method ini menampilkan data mahasiswa dalam format:

nama - nim - prodi

Contoh hasil output:

Naysha - 2511531014 - Informatika

9. Pada method `toString()`, digunakan annotation `@Override` yang menandakan bahwa method tersebut menggantikan method bawaan dari class `Object`. Dengan demikian, saat objek dipanggil menggunakan `System.out.println()`, data mahasiswa dapat langsung ditampilkan dalam format yang lebih rapi.
10. Secara keseluruhan, program ini digunakan untuk memahami konsep dasar class dan object dalam Java, termasuk penggunaan atribut, constructor, getter, setter, dan method `toString()`. Program ini juga menerapkan konsep enkapsulasi karena data disimpan dalam atribut `private` dan diakses melalui method tertentu. Dengan implementasi ini, pengelolaan data mahasiswa menjadi lebih terstruktur, aman, dan mudah dikembangkan.

```

1  package tugaspekan7_2511531014;
2
3  import java.awt.BorderLayout;
4  import java.awt.Dimension;
5  import java.awt.EventQueue;
6  import java.awt.FlowLayout;
7  import java.awt.Font;
8  import java.awt.GridLayout;
9  import java.awt.event.ActionEvent;
10 import java.awt.event.ActionListener;
11 import java.util.ArrayList;
12
13 import javax.swing.BorderFactory;
14 import javax.swing.JButton;
15 import javax.swing.JComboBox;
16 import javax.swing.JFrame;
17 import javax.swing.JPanel;
18 import javax.swing.JScrollPane;
19 import javax.swing.JTable;
20 import javax.swing.JTextArea;
21 import javax.swing.border.EmptyBorder;
22 import javax.swing.table.DefaultTableModel;
23 import javax.swing.JLabel;
24 import javax.swing.JOptionPane;
25 import javax.swing.JTextField;
26
27 public class SortingGUI_2511531014 extends JFrame {
28
29     private static final long serialVersionUID = 1L;
30     private JPanel contentPane;
31
32     // DEKLARASI KOMPONEN GUI
33     private JTextField txtNama_1014, txtNim_1014, txtProdi_1014;
34     private JButton btnTambah_1014, btnHapus_1014, btnSort_1014;
35     private JComboBox<String> cmbAlgoritma_1014;
36     private JTable tabelData_1014;
37     private DefaultTableModel tableModel_1014;
38     private JTextArea txtAreaLog_1014;
39
40     // ArrayList penyimpanan data mahasiswa
41     private ArrayList<Mahasiswa_2511531014> listMahasiswa;
42
43     // CONSTRUCTOR
44     public SortingGUI_2511531014() {
45
46         setTitle("Pengurutan Nama Mahasiswa - Pekan 7");
47         setSize(850, 650);
48         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
49         setLocationRelativeTo(null);
50

```

```

50
51     listMahasiswa = new ArrayList<>();
52
53     contentPane = new JPanel();
54     contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
55     setContentPane(contentPane);
56     contentPane.setLayout(new BorderLayout(15, 15));
57
58     // PANEL INPUT
59     JPanel panelInput = new JPanel(new GridLayout(4, 2, 10, 10));
60
61     panelInput.setBorder(
62         BorderFactory.createTitledBorder(
63             "Form Input Mahasiswa"));
64
65     panelInput.add(new JLabel("NIM Mahasiswa:"));
66
67     txtNim_1014 = new JTextField();
68     panelInput.add(txtNim_1014);
69
70     panelInput.add(new JLabel("Nama Mahasiswa:"));
71
72     txtNama_1014 = new JTextField();
73     panelInput.add(txtNama_1014);
74
75     panelInput.add(new JLabel("Program Studi:"));
76
77     txtProdi_1014 = new JTextField();
78     panelInput.add(txtProdi_1014);
79
80     btnTambah_1014 = new JButton("Tambah Data");
81     btnHapus_1014 = new JButton("Kosongkan Semua Data");
82
83     panelInput.add(btnTambah_1014);
84     panelInput.add(btnHapus_1014);
85
86     contentPane.add(panelInput, BorderLayout.NORTH);
87
88     // PANEL TENGAH
89     JPanel panelTengah =
90         new JPanel(new BorderLayout(10, 10));
91
92     JPanel panelSaring =
93         new JPanel(new FlowLayout(FlowLayout.LEFT));
94
95     panelSaring.add(
96         new JLabel("Pilih Algoritma Sorting:"));
97
98     String[] pilihanAlgoritma = {
99         "Insertion Sort",
100         "Selection Sort",

```

```

101         "Bubble Sort"
102     };
103
104     cmbAlgoritma_1014 =
105         new JComboBox<>(pilihanAlgoritma);
106
107     panelSaring.add(cmbAlgoritma_1014);
108
109     btnSort_1014 =
110         new JButton("Mulai Sorting");
111
112     panelSaring.add(btnSort_1014);
113
114     panelTengah.add(panelSaring, BorderLayout.NORTH);
115
116     // TABEL
117     String[] kolom = {
118         "NIM",
119         "Nama Mahasiswa",
120         "Program Studi"
121     };
122
123     tableModel_1014 =
124         new DefaultTableModel(kolom, 0);
125
126     tabelData_1014 =
127         new JTable(tableModel_1014);
128
129     JScrollPane scrollTabel =
130         new JScrollPane(tabelData_1014);
131
132     scrollTabel.setPreferredSize(
133         new Dimension(800, 150));
134
135     panelTengah.add(scrollTabel,
136         BorderLayout.CENTER);
137
138     contentPane.add(panelTengah,
139         BorderLayout.CENTER);
140
141     // PANEL LOG
142     JPanel panelBawah =
143         new JPanel(new BorderLayout());
144
145     panelBawah.setBorder(
146         BorderFactory.createTitledBorder(
147             "Visualisasi Proses Langkah demi Langkah"));
148
149     txtAreaLog_1014 = new JTextArea();
150
151     txtAreaLog_1014.setEditable(false);

```

```

151     txtAreaLog_1014.setEditable(false);
152
153     txtAreaLog_1014.setFont(
154         new Font("Monospaced",
155             Font.PLAIN, 13));
156
157     JScrollPane scrollLog =
158         new JScrollPane(txtAreaLog_1014);
159
160     scrollLog.setPreferredSize(
161         new Dimension(800, 220));
162
163     panelBawah.add(scrollLog,
164         BorderLayout.CENTER);
165
166     contentPane.add(panelBawah,
167         BorderLayout.SOUTH);
168
169     // EVENT BUTTON TAMBAH
170     btnTambah_1014.addActionListener(
171         new ActionListener() {
172
173             public void actionPerformed(
174                 ActionEvent e) {
175
176                 tambahDataKeList();
177             }
178         });
179
180     // EVENT BUTTON HAPUS
181     btnHapus_1014.addActionListener(
182         new ActionListener() {
183
184             @Override
185             public void actionPerformed(
186                 ActionEvent e) {
187
188                 listMahasiswa.clear();
189
190                 refreshTabel();
191
192                 txtAreaLog_1014.setText("");
193             }
194         });
195
196     // EVENT BUTTON SORT
197     btnSort_1014.addActionListener(
198         new ActionListener() {
199
200             @Override

```

```

201         public void actionPerformed(
202             ActionEvent e) {
203             eksekusiSorting();
204         }
205     });
206 }
207
208 // METHOD INPUT DATA
209 private void tambahDataKeList() {
210     String nim =
211         txtNim_1014.getText().trim();
212     String nama =
213         txtNama_1014.getText().trim();
214     String prodi =
215         txtProdi_1014.getText().trim();
216
217     if (nim.isEmpty() ||
218         nama.isEmpty() ||
219         prodi.isEmpty()) {
220         JOptionPane.showMessageDialog(
221             this,
222             "Isi semua data terlebih dahulu!",
223             "Peringatan",
224             JOptionPane.WARNING_MESSAGE);
225         return;
226     }
227
228     Mahasiswa_2511531014 mhs =
229         new Mahasiswa_2511531014(
230             nama, nim, prodi);
231
232     listMahasiswa.add(mhs);
233
234     refreshTabel();
235
236     txtNim_1014.setText("");
237     txtNama_1014.setText("");
238     txtProdi_1014.setText("");
239
240     txtNim_1014.requestFocus();
241 }
242
243 // METHOD REFRESH TABEL
244 private void refreshTabel() {
245

```

```

250 private void refreshLabel() {
251
252     tableModel_1014.setRowCount(0);
253
254     for (Mahasiswa_2511531014 mhs
255          : listMahasiswa) {
256
257         tableModel_1014.addRow(
258             new Object[] {
259                 mhs.getNim(),
260                 mhs.getNama(),
261                 mhs.getProdi()
262             });
263     }
264 }
265
266 // METHOD SORTING
267 private void eksekusiSorting() {
268
269     if (listMahasiswa.size() < 2) {
270
271         JOptionPane.showMessageDialog(
272             this,
273             "Masukkan minimal 2 data mahasiswa!",
274             "Info",
275             JOptionPane.INFORMATION_MESSAGE);
276
277         return;
278     }
279
280     ArrayList<Mahasiswa_2511531014>
281         dataUrut =
282             new ArrayList<>(listMahasiswa);
283
284     txtAreaLog_1014.setText("");
285
286     String algoritmaTerpilih =
287         (String)
288         cmbAlgoritma_1014.getSelectedItem();
289
290     if (algoritmaTerpilih.equals(
291         "Insertion Sort")) {
292
293         jalankanInsertionSort(dataUrut);
294
295     } else if (algoritmaTerpilih.equals(
296         "Selection Sort")) {
297
298         jalankanSelectionSort(dataUrut);
299
300     } else if (algoritmaTerpilih.equals(

```

```

301         "Bubble Sort")) {
302
303             jalankanBubbleSort(dataUrut);
304         }
305
306         listMahasiswa = dataUrut;
307
308         refreshTabel();
309     }
310
311     // INSERTION SORT
312     private void jalankanInsertionSort(
313         ArrayList<Mahasiswa_2511531014> arr) {
314
315         txtAreaLog_1014.append(
316             "=== INSERTION SORT ===\n");
317
318         txtAreaLog_1014.append(
319             "Awal : " +
320             arr.toString() + "\n\n");
321
322         int n = arr.size();
323
324         for (int i = 1; i < n; i++) {
325
326             Mahasiswa_2511531014 key =
327                 arr.get(i);
328
329             int j = i - 1;
330
331             while (j >= 0 &&
332                 arr.get(j).getNama()
333                     .compareToIgnoreCase(
334                         key.getNama()) > 0) {
335
336                 arr.set(j + 1,
337                     arr.get(j));
338
339                 j = j - 1;
340             }
341
342             arr.set(j + 1, key);
343
344             txtAreaLog_1014.append(
345                 "Langkah " + i +
346                 " : " +
347                 arr.toString() + "\n");
348         }
349
350         txtAreaLog_1014.append(

```

```

351         "\nStatus: Pengurutan Berhasil!");
352     }
353
354     // SELECTION SORT
355     private void jalankanSelectionSort(
356         ArrayList<Mahasiswa_2511531014> arr) {
357
358         txtAreaLog_1014.append(
359             "=== SELECTION SORT ===\n");
360
361         txtAreaLog_1014.append(
362             "Awal : " +
363             arr.toString() + "\n\n");
364
365         int n = arr.size();
366
367         for (int i = 0; i < n - 1; i++) {
368             int indeksMinimal = i;
369
370             for (int j = i + 1;
371                 j < n; j++) {
372
373                 if (arr.get(j).getNama()
374                     .compareToIgnoreCase(
375                         arr.get(indeksMinimal)
376                             .getNama()) < 0) {
377
378                     indeksMinimal = j;
379                 }
380             }
381
382             Mahasiswa_2511531014 temp =
383                 arr.get(indeksMinimal);
384
385             arr.set(indeksMinimal,
386                 arr.get(i));
387
388             arr.set(i, temp);
389
390             txtAreaLog_1014.append(
391                 "Pass " + (i + 1) +
392                 " : " +
393                 arr.toString() + "\n");
394         }
395
396         txtAreaLog_1014.append(
397             "\nStatus: Pengurutan Berhasil!");
398     }
399
400

```

```

400
401 // BUBBLE SORT
402 private void jalankanBubbleSort(
403     ArrayList<Mahasiswa_2511531014> arr) {
404
405     txtAreaLog_1014.append(
406         "=== BUBBLE SORT ===\n");
407
408     txtAreaLog_1014.append(
409         "Awal : " +
410         arr.toString() + "\n\n");
411
412     int n = arr.size();
413
414     for (int i = 0; i < n - 1; i++) {
415
416         for (int j = 0;
417             j < n - i - 1; j++) {
418
419             if (arr.get(j).getNama()
420                 .compareToIgnoreCase(
421                     arr.get(j + 1)
422                     .getNama()) > 0) {
423
424                 Mahasiswa_2511531014 temp =
425                     arr.get(j);
426
427                 arr.set(j,
428                     arr.get(j + 1));
429
430                 arr.set(j + 1,
431                     temp);
432             }
433         }
434
435         txtAreaLog_1014.append(
436             "Pass " + (i + 1) +
437             " : " +
438             arr.toString() + "\n");
439     }
440
441     txtAreaLog_1014.append(
442         "\nStatus: Pengurutan Berhasil!");
443 }
444
445 // MAIN METHOD
446 public static void main(String[] args) {
447
448     EventQueue.invokeLater(
449         new Runnable() {
450
451             public void run() {
452
453                 try {
454
455                     SortingGUI_2511531014 frame =
456                         new SortingGUI_2511531014();
457
458                     frame.setVisible(true);
459
460                 } catch (Exception e) {
461
462                     e.printStackTrace();
463                 }
464             }
465         });
466 }
467 }

```

### • Pembahasan

Program ini dibuat dengan mendefinisikan class SortingGUI\_2511531014 yang digunakan untuk mengimplementasikan algoritma sorting secara visual menggunakan Java GUI (Swing). Program ini memungkinkan pengguna untuk memasukkan data mahasiswa, menampilkan data ke dalam tabel, serta melakukan proses pengurutan nama mahasiswa menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort secara interaktif.

1. Class `SortingGUI_2511531014` merupakan turunan dari `JFrame` yang digunakan sebagai window utama pada aplikasi GUI. Program memanfaatkan berbagai komponen Swing seperti `JPanel`, `JButton`, `TextField`, `JTable`, `JComboBox`, `TextArea`, dan `JScrollPane` untuk membangun tampilan antarmuka program.
2. Program memiliki atribut `txtNama_1014`, `txtNim_1014`, dan `txtProdi_1014` yang digunakan sebagai field input data mahasiswa. Selain itu terdapat `JTable` `tabelData_1014` yang digunakan untuk menampilkan data mahasiswa dalam bentuk tabel.
3. Program menggunakan `ArrayList<Mahasiswa_2511531014>` `listMahasiswa` sebagai tempat penyimpanan data mahasiswa secara dinamis. Setiap data mahasiswa disimpan dalam bentuk objek dari class `Mahasiswa_2511531014`.
4. Pada constructor `SortingGUI_2511531014()`, program mengatur tampilan utama GUI seperti ukuran window, judul program, layout, serta menambahkan seluruh komponen GUI ke dalam frame. Constructor juga digunakan untuk menginisialisasi `ArrayList` dan mengatur event button.
5. Panel input dibuat menggunakan `GridLayout` yang berisi field NIM, nama mahasiswa, dan program studi. Pada panel ini juga terdapat tombol Tambah Data dan Kosongkan Semua Data. Tombol tambah digunakan untuk memasukkan data mahasiswa ke dalam list, sedangkan tombol hapus digunakan untuk menghapus seluruh data mahasiswa yang tersimpan.
6. Program memiliki `JComboBox` `cmbAlgoritma_1014` yang digunakan untuk memilih algoritma sorting yang ingin dijalankan. Pilihan algoritma yang tersedia adalah Insertion Sort, Selection Sort, dan Bubble Sort.
7. Data mahasiswa ditampilkan menggunakan `JTable` dengan `DefaultTableModel`. Tabel memiliki tiga kolom yaitu NIM, Nama Mahasiswa, dan Program Studi. Method `refreshTabel()` digunakan untuk memperbarui isi tabel sesuai data terbaru pada `ArrayList`.
8. Program juga memiliki `TextArea` `txtAreaLog_1014` yang digunakan untuk menampilkan proses langkah demi langkah sorting. Hasil setiap proses sorting akan ditampilkan dalam area log sehingga pengguna dapat melihat perubahan posisi data secara bertahap.
9. Method `tambahDataKeList()` digunakan untuk mengambil input dari `TextField`, kemudian membuat objek mahasiswa baru dan menyimpannya ke dalam `ArrayList`. Method ini juga melakukan validasi agar tidak ada field yang kosong. Jika input kosong, program akan menampilkan pesan peringatan menggunakan `JOptionPane`.

10. Method `eksekusiSorting()` digunakan untuk menjalankan proses sorting berdasarkan algoritma yang dipilih pada `JComboBox`. Program akan memeriksa pilihan pengguna kemudian memanggil method sorting yang sesuai, yaitu `jalankanInsertionSort()`, `jalankanSelectionSort()`, atau `jalankanBubbleSort()`.
11. Method `jalankanInsertionSort()` digunakan untuk mengurutkan data mahasiswa menggunakan algoritma Insertion Sort berdasarkan nama mahasiswa. Program membandingkan nama menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf besar dan kecil. Setiap langkah sorting ditampilkan pada `JTextArea`.
12. Method `jalankanSelectionSort()` digunakan untuk mengurutkan data menggunakan algoritma Selection Sort. Program mencari nama mahasiswa dengan urutan alfabet paling kecil kemudian menukarnya dengan posisi yang sesuai. Setiap proses pertukaran data ditampilkan pada log langkah sorting.
13. Method `jalankanBubbleSort()` digunakan untuk melakukan pengurutan dengan cara membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Proses ini dilakukan berulang hingga seluruh data terurut. Hasil setiap pass ditampilkan pada area log GUI.
14. Program menggunakan method `compareToIgnoreCase()` untuk membandingkan nama mahasiswa secara alfabetis tanpa memperhatikan huruf kapital dan huruf kecil sehingga hasil pengurutan menjadi lebih konsisten.
15. Pada method `main`, program dijalankan menggunakan `EventQueue.invokeLater()` agar proses pembuatan GUI berjalan pada event dispatch thread sehingga tampilan GUI dapat berjalan dengan stabil dan aman.
16. Secara keseluruhan, program ini digunakan untuk memahami implementasi algoritma sorting dalam bentuk GUI interaktif menggunakan Java Swing. Pengguna dapat menambahkan data mahasiswa, memilih algoritma sorting, melihat proses pengurutan langkah demi langkah, serta memahami perbedaan cara kerja Insertion Sort, Selection Sort, dan Bubble Sort secara visual dan lebih mudah dipahami.

## HASIL OUTPUT

Pengurutan Nama Mahasiswa - Pekan 7

**Form Input Mahasiswa**

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Pilih Algoritma Sorting:

| NIM  | Nama Mahasiswa | Program Studi |
|------|----------------|---------------|
| 1014 | naysha         | struktur data |
| 3004 | silva          | struktur data |
| 3030 | aisyah         | struktur data |

**Visualisasi Proses Langkah demi Langkah**

Pengurutan Nama Mahasiswa - Pekan 7

**Form Input Mahasiswa**

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Pilih Algoritma Sorting:

| NIM  | Nama Mahasiswa | Program Studi |
|------|----------------|---------------|
| 3030 | aisyah         | struktur data |
| 1014 | naysha         | struktur data |
| 3004 | silva          | struktur data |

**Visualisasi Proses Langkah demi Langkah**

```
=== INSERTION SORT ===
Awal : [naysha - 1014 - struktur data, silva - 3004 - struktur data, aisyah - 3030 - struktur data]

Langkah 1 : [naysha - 1014 - struktur data, silva - 3004 - struktur data, aisyah - 3030 - struktur data]
Langkah 2 : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]

Status: Pengurutan Berhasil!
```

Pengurutan Nama Mahasiswa - Pekan 7

**Form Input Mahasiswa**

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Pilih Algoritma Sorting: Selection Sort

| NIM  | Nama Mahasiswa | Program Studi |
|------|----------------|---------------|
| 3030 | aisyah         | struktur data |
| 1014 | naysha         | struktur data |
| 3004 | silva          | struktur data |

**Visualisasi Proses Langkah demi Langkah**

```
=== SELECTION SORT ===  
Awal : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
  
Pass 1 : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
Pass 2 : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
  
Status: Pengurutan Berhasil!
```

Pengurutan Nama Mahasiswa - Pekan 7

**Form Input Mahasiswa**

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Pilih Algoritma Sorting: Bubble Sort

| NIM  | Nama Mahasiswa | Program Studi |
|------|----------------|---------------|
| 3030 | aisyah         | struktur data |
| 1014 | naysha         | struktur data |
| 3004 | silva          | struktur data |

**Visualisasi Proses Langkah demi Langkah**

```
=== BUBBLE SORT ===  
Awal : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
  
Pass 1 : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
Pass 2 : [aisyah - 3030 - struktur data, naysha - 1014 - struktur data, silva - 3004 - struktur data]  
  
Status: Pengurutan Berhasil!
```